

第四部分：从零开始的深度学习

17. 神经网络基础

本章开启一段旅程，我们将深入剖析前几章所用模型的内部机制。虽然涉及许多熟悉的内容，但这次我们将更聚焦于实现细节，而非实际应用层面的“如何实现”与“为何如此设计”。

我们将从零开始构建所有内容，仅使用张量的基本索引操作。我们将从头编写神经网络，然后手动实现反向传播，以便在调用 `loss.backward` 时精确理解 PyTorch 内部的运作机制。我们还将探索如何通过自定义自动微分函数扩展 PyTorch，从而能够指定专属的前向与反向计算流程。

从零开始构建神经网络层

让我们先重新梳理矩阵乘法在基础神经网络中的应用原理。由于我们将从零开始构建所有内容，初期仅使用纯Python实现（除PyTorch张量索引操作外），待掌握创建方法后再用PyTorch功能替换纯Python代码。

神经元建模

一个神经元接收特定数量的输入，并对每个输入赋予内部权重。它将这些加权输入相加产生输出，并添加一个内部偏置。在数学上，这可表示为：

$$out = \sum_{i=1}^n x_i w_i + b$$

若我们将输入命名为 $(x_1, \dots, x_n)$ ，权重命名为 $(w_1, \dots, w_n)$ ，偏置项命名为 b 。在代码中对应如下：

```
output = sum([x*w for x,w in zip(inputs,weights)]) + bias
```

该输出随后被输入到称为激活函数的非线性函数中，再传递至另一个神经元。在深度学习中，最常见的激活函数是ReLU，正如我们所见，这其实是这样一种表达方式：

```
def relu(x): return x if x >= 0 else 0
```

随后通过将大量神经元按顺序分层堆叠，构建出深度学习模型。我们创建一个包含特定数量神经元（称为隐藏层大小）的第一层，并将所有输入连接至该层的每个神经元。此类层通常被称为全连接层、稠密层（因其紧密连接特性）或线性层。

它要求你针对每个 `input` 和每个具有给定 `weight` 的神经元，计算点积：

```
sum([x*w for x,w in zip(input,weight)])
```

如果你学过一点线性代数，可能会记得，当进行矩阵乘法时，会出现大量点积运算。更精确地说，若输入数据存储在矩阵 `x` 中，其尺寸为 `batch_size x n_inputs`；若我们将神经元的权重归类到矩阵 `w` 中，其尺寸为 `n_neurons x n_inputs`（每个神经元的权重数量必须与其输入数量相同）；并将所有偏置值存储在向量 `b` 中，其尺寸为 `n_neurons`，那么该全连接层的输出为：

```
y = x @ w.t() + b
```

其中 `@` 表示矩阵乘法, `w.t()` 是 `w` 的转置矩阵。输出 `y` 的尺寸为 `batch_size × n_neurons`, 在位置 `(i,j)` 处我们得到如下结果 (供数学爱好者参考) :

$$y_{i,j} = \sum_{k=1}^n x_{i,k} w_{k,j} + b_j$$

或者用代码表示:

```
y[i,j] = sum([a * b for a,b in zip(x[i,:],w[j,:])]) + b[j]
```

转置是必要的, 因为在矩阵乘法 `m @ n` 的数学定义中, 系数 `(i,j)` 的形式如下:

```
sum([a * b for a,b in zip(m[i,:],n[:,j])])
```

因此我们需要的基本运算就是矩阵乘法, 因为它正是神经网络核心中隐藏的本质。

从零开始的矩阵乘法

在允许使用PyTorch版本之前, 让我们先编写一个计算两个张量矩阵乘积的函数。我们将仅使用PyTorch张量中的索引功能:

```
import torch
from torch import tensor
```

我们需要三个嵌套的 `for` 循环: 一个用于行索引, 一个用于列索引, 一个用于内部求和。 `ac` 和 `ar` 分别表示 `a` 的列数和 `a` 的行数 (`b` 也遵循相同约定), 并通过检查 `a` 的列数是否等于 `b` 的行数来确保矩阵乘法计算的可行性:

```
def matmul(a,b):
    ar,ac = a.shape # n_rows * n_cols
    br,bc = b.shape
    assert ac==br
    c = torch.zeros(ar, bc)
    for i in range(ar):
        for j in range(bc):
            for k in range(ac): c[i,j] += a[i,k] * b[k,j]
    return c
```

为验证此方法, 我们将假设 (使用随机矩阵) 处理一个包含5张MNIST图像的小批量数据集, 这些图像被展平为 `28×28` 的向量, 并通过线性模型将其转换为10个激活值:

```
m1 = torch.randn(5,28*28)
m2 = torch.randn(784,10)
```

让我们使用Jupyter的“魔法”命令 `%time` 来计时我们的函数:

```
%time t1=matmul(m1, m2)
```

```
CPU times: user 1.15 s, sys: 4.09 ms, total: 1.15 s
wall time: 1.15 s
```

再看看这与PyTorch内置的 `@` 相比如何？

```
%timeit -n 20 t2=m1@m2
```

```
14 µs ± 8.95 µs per loop (mean ± std. dev. of 7 runs, 20  
loops each)
```

如我们所见，在Python中使用三重嵌套循环是个糟糕的主意！Python本就是一种运行缓慢的语言，这种写法效率极低。我们发现PyTorch的速度比Python快约10万倍——这还是在尚未启用GPU加速的情况下！

这种差异从何而来？PyTorch并未用Python实现矩阵乘法，而是采用C++实现以提升速度。通常，当我们对张量进行计算时，需要将其向量化以充分发挥PyTorch的速度优势，这通常通过两种技术实现：元素级运算和广播。

元素级运算

所有基本运算符（`+`，`-`，`*`，`/`，`>`，`<`，`==`）均可进行元素级运算。这意味着当两个张量 `a` 和 `b` 具有相同形状时，若写为 `a+b`，则会得到一个由 `a` 和 `b` 元素之和组成的张量：

```
a = tensor([10., 6, -4])  
b = tensor([2., 8, 7])  
a + b
```

```
tensor([12., 14., 3.])
```

布尔运算符将返回一个布尔值数组：

```
a < b
```

```
tensor([False, True, True])
```

若要判断向量 `a` 的每个元素是否都小于向量 `b` 的对应元素，或判断两个张量是否相等，我们需要将这些元素级运算与 `torch.all` 函数结合使用：

```
(a < b).all(), (a==b).all()
```

```
(tensor(False), tensor(False))
```

缩减操作（如 `all`、`sum` 和 `mean`）会返回仅含一个元素的张量，称为零阶张量。若需将其转换为普通的Python布尔值或数字，需调用 `.item`：

```
(a + b).mean().item()
```

```
9.6666666984558105
```

元素级运算适用于任意阶张量，只要它们具有相同的形状：

```
m = tensor([[1., 2, 3], [4,5,6], [7,8,9]])
m*m
```

```
tensor([[ 1., 4., 9.],
        [16., 25., 36.],
        [49., 64., 81.]])
```

然而，对于形状不相同的张量，无法执行元素级运算（除非它们可广播，详见下一节讨论）：

```
n = tensor([[1., 2, 3], [4,5,6]])
m*n
```

```
RuntimeError: The size of tensor a (3) must match the size
of tensor b (2) at
dimension 0
```

通过元素级运算，我们可以移除三个嵌套循环中的一个：在求和所有元素之前，先将对应于 `a` 的第 `i` 行与 `b` 的第 `j` 列的张量相乘，这将提高计算速度，因为此时内部循环将由 PyTorch 以 C 语言的速度执行。

要访问某一列或某一行，只需编写 `a[i,:]` 或 `b[:,j]`。冒号表示取该维度上的所有元素。我们也可以通过传递范围（如 `1:5`）来限制取值范围，而非仅使用冒号。此时将取第 1 至第 4 列的元素（第二个数字不包含该列）。

一种简化方式是始终省略尾随冒号，因此 `a[i,:]` 可简写为 `a[i]`。基于上述原则，我们可以重写矩阵乘法的新版本：

```
def matmul(a,b):
    ar,ac = a.shape
    br,bc = b.shape
    assert ac==br
    c = torch.zeros(ar, bc)
    for i in range(ar):
        for j in range(bc): c[i,j] = (a[i] * b[:,j]).sum()
    return c
```

```
%timeit -n 20 t3 = matmul(m1,m2)
```

```
1.7 ms ± 88.1 µs per loop (mean ± std. dev. of 7 runs, 20
loops each)
```

仅通过移除内部 `for` 循环，我们的速度已提升约 700 倍！而这仅仅是开始——借助广播功能，我们还能移除另一个循环，实现更显著的加速效果。

广播

正如我们在第4章所讨论的，广播是 [Numpy库](#) 引入的一个术语，用于描述不同阶数张量在算术运算中的处理方式。例如，显然无法将 3×3 矩阵与 4×5 矩阵相加，但若要将一个标量（可表示为 1×1 张量）与矩阵相加呢？或者将 3 维向量与 3×4 矩阵相加呢？这两种情况下，我们都能找到使该运算成立的方法。

广播机制提供了具体规则，用于规范元素级运算中形状的兼容性，以及如何扩展较小形状的张量以匹配较大形状的张量。若想编写高效运行的代码，掌握这些规则至关重要。本节将扩展我们对广播机制的先前讨论，深入理解这些规则。

使用标量进行广播

标量广播是最简单的广播类型。当我们有一个张量 `a` 和一个标量时，只需想象一个与 `a` 形状相同的张量，其中填充着该标量，然后执行运算：

```
a = tensor([10., 6, -4])
a > 0
```

```
tensor([ True,  True, False])
```

我们如何实现这种比较？`0` 被广播为与 `a` 具有相同的维度。请注意，此操作无需在内存中创建一个充满零的张量（那样效率低下）。

若需对数据集进行标准化处理，可通过以下方式实现：从整个数据集（矩阵）中减去均值（标量），再除以标准差（另一个标量）：

```
m = tensor([[1., 2, 3], [4,5,6], [7,8,9]])
(m - 5) / 2.73
```

```
tensor([[ -1.4652, -1.0989, -0.7326],
        [-0.3663,  0.0000,  0.3663],
        [ 0.7326,  1.0989,  1.4652]])
```

如果矩阵的每行都有不同的运算方式呢？在这种情况下，你需要将向量广播为矩阵。

将向量广播为矩阵

我们可以按以下方式将向量广播为矩阵：

```
c = tensor([10.,20,30])
m = tensor([[1., 2, 3], [4,5,6], [7,8,9]])
m.shape,c.shape
```

```
(torch.Size([3, 3]), torch.Size([3]))
```

```
m + c
```

```
tensor([[11., 22., 33.],
        [14., 25., 36.],
        [17., 28., 39.]])
```

此处将 `c` 的元素展开为三行匹配数据，使运算得以实现。同样地，PyTorch并未在内存中实际创建 `c` 的三个副本。这由后台的 `expand_as` 方法完成：

```
c.expand_as(m)
```

```
tensor([[10., 20., 30.],
        [10., 20., 30.],
        [10., 20., 30.]])
```

若考察对应的张量，可通过其存储属性（该属性展示张量实际占用的内存内容）来验证是否存在无用数据：

```
t = c.expand_as(m)
t.storage()
```

```
10.0
20.0
30.0
[torch.FloatTensor of size 3]
```

尽管张量官方定义有九个元素，但是内存中仅存储了三个标量。这得益于一个巧妙的技巧：为该维度设置步长为0（这意味着当PyTorch通过累加步长寻找下一行时，该维度不会移动）：

```
t.stride(), t.shape
```

```
((0, 1), torch.Size([3, 3]))
```

由于 `m` 是 3×3 的张量，广播操作有两种方式。选择在最后维度进行广播是广播规则的约定惯例，与张量排列顺序无关。若采用以下方式操作，结果相同：

```
c + m
```

```
tensor([[11., 22., 33.],
        [14., 25., 36.],
        [17., 28., 39.]])
```

事实上，仅能使用 $m \times n$ 矩阵来广播 n 维向量：

```
c = tensor([10., 20, 30])
m = tensor([[1., 2, 3], [4, 5, 6]])
c+m
```

```
tensor([[11., 22., 33.],
        [14., 25., 36.]])
```

这行不通：

```
c = tensor([10., 20])
m = tensor([[1., 2, 3], [4, 5, 6]])
c+m
```

```
RuntimeError: The size of tensor a (2) must match the size
of tensor b (3) at
dimension 1
```

若要在另一维度进行广播，我们需要将向量的形状转换为3×1矩阵。这可通过PyTorch的 `unsqueeze` 方法实现：

```
c = tensor([10., 20, 30])
m = tensor([[1., 2, 3], [4, 5, 6], [7, 8, 9]])
c = c.unsqueeze(1)
m.shape, c.shape
```

```
(torch.Size([3, 3]), torch.Size([3, 1]))
```

这次，`c` 在列侧展开：

```
c+m
```

```
tensor([[11., 12., 13.],
        [24., 25., 26.],
        [37., 38., 39.]])
```

与之前相同，内存中仅存储三个标量：

```
t = c.expand_as(m)
t.storage()
```

```
10.0
20.0
30.0
[torch.FloatTensor of size 3]
```

扩展后的张量具有正确的形状，因为列维度的步长为0：

```
t.stride(), t.shape
```

```
((1, 0), torch.Size([3, 3]))
```

在广播操作中，若需添加维度，默认会在开头添加。此前进行广播时，PyTorch 实际上在后台执行了 `c.unsqueeze(0)` 操作：

```
c = tensor([10., 20, 30])
c.shape, c.unsqueeze(0).shape, c.unsqueeze(1).shape
```

```
(torch.Size([3]), torch.Size([1, 3]), torch.Size([3, 1]))
```

`uncompress` 命令可通过 `None` 索引替代：

```
c.shape, c[None,:].shape, c[:,None].shape
```

```
(torch.Size([3]), torch.Size([1, 3]), torch.Size([3, 1]))
```

尾随冒号可省略，而 `...` 表示所有前置维度

```
c[None].shape, c[... ,None].shape
```

```
(torch.Size([1, 3]), torch.Size([3, 1]))
```

这样，我们就能够在矩阵乘法函数中移除另一个 `for` 循环。现在，无需将 `a[i]` 与 `b[:,j]` 相乘，而是通过广播操作将 `a[i]` 与整个矩阵 `b` 相乘，然后求和结果：

```
def matmul(a,b):
    ar,ac = a.shape
    br,bc = b.shape
    assert ac==br
    c = torch.zeros(ar, bc)
    for i in range(ar):
        # c[i,j] = (a[i,:] * b[:,j]).sum() # previous
        c[i] = (a[i].unsqueeze(-1) * b).sum(dim=0)
    return c
```

```
%timeit -n 20 t4 = matmul(m1,m2)
```

```
357 µs ± 7.2 µs per loop (mean ± std. dev. of 7 runs, 20
loops each)
```

我们现在的速度比最初实现快了3700倍！在继续之前，让我们更详细地讨论广播规则。

广播规则

当对两个张量进行操作时，PyTorch会逐元素比较它们的形状。它从尾部维度开始，逐步向头部推进，遇到空维度时加1。当满足以下任一条件时，两个维度即为兼容（compatible）：

- 它们是相等的。
- 其中一个为1，此时该维度将被广播使其与另一个相同。

数组不必具有相同的维数。例如，若有一个256×256×3的RGB值数组，且需要对图像中每种颜色应用不同的缩放系数，则可将该图像与一个包含三个值的一维数组相乘。根据广播规则对齐这些数组末尾轴的尺寸后，可验证它们是兼容的：

```
Image (3d tensor): 256 x 256 x 3
Scale (1d tensor): (1) (1) 3
Result (3d tensor): 256 x 256 x 3
```

然而，一个尺寸为256×256的二维张量与我们的图像不兼容：


```
Image (3d tensor): 256 x 256 x 3
Scale (1d tensor): (1) 256 x 256
Error
```

在我们之前关于3×3矩阵和3维向量的示例中，广播操作是在行方向上进行的：

```
Matrix (2d tensor): 3 x 3
Vector (1d tensor): (1) 3
Result (2d tensor): 3 x 3
```

作为练习，请尝试确定需要添加哪些维度（以及添加位置），当你需要对一批尺寸为 64×3×256×256 的图像进行归一化处理时，这些图像对应的向量包含三个元素（一个表示均值，另一个表示标准差）。

简化张量运算的另一种有效方法是采用爱因斯坦求和约定。

爱因斯坦求和

在使用 PyTorch 的 `@` 运算符或 `torch.matmul` 之前，我们还有一种最后的矩阵乘法实现方式：爱因斯坦求和（`einsum`）。这是将乘积与求和以通用方式组合的紧凑表示法。我们写出这样的方程：

```
ik,kj -> ij
```

左侧表示操作数的维度，用逗号分隔。这里有两个张量，每个张量各有两个维度（`i,k` 和 `k,j`）。右侧表示结果的维度，因此这里得到一个具有两个维度 `i,j` 的张量。

爱因斯坦求和符号的规则如下：

1. 重复的下标将隐含地进行求和。
2. 每个下标在任何项中最多出现两次。
3. 每个项必须包含相同的非重复下标。

因此在我们的例子中，由于 `k` 是重复的，我们需要对该索引求和。最终，该公式表示当我们将 `(i,j)` 填入时所得到的矩阵——即第一个张量中所有系数 `(i,k)` 与第二个张量中系数 `(k,j)` 相乘的和……这正是矩阵乘法！

以下是在 PyTorch 中实现此功能的代码：

```
def matmul(a,b): return torch.einsum('ik,kj->ij', a, b)
```

爱因斯坦求和是一种表达涉及索引和积和的运算的非常实用的方法。请注意，左侧可以只有一个元素。例如：

```
torch.einsum('ij->ji', a)
```

它返回矩阵 `a` 的转置。你也可以拥有三个或更多的成员：

```
torch.einsum('bi,ij,bj->b', a, b, c)
```

这将返回一个大小为 `b` 的向量，其中第 `k` 个坐标是 `a[k,i]`、`b[i,j]` 和 `c[k,j]` 的和。当因批处理而存在更多维度时，这种表示法尤为便捷。例如，若你有两批矩阵并希望按批次计算矩阵乘积，可采用以下方式：

```
torch.einsum('bik,bkj->bij', a, b)
```

让我们回到使用 `einsum` 实现的新 `matmul` 实现，并看看它的速度：

```
%timeit -n 20 t5 = matmul(m1,m2)
```

```
68.7 µs ± 4.06 µs per loop (mean ± std. dev. of 7 runs, 20
loops each)
```

如你所见，它不仅实用，而且速度极快。`einsum` 通常是PyTorch中执行自定义操作最快捷的方式，无需深入C++和CUDA。（但正如你在《从零开始实现矩阵乘法》中的结果所见，它通常不如精心优化的CUDA代码高效。）

既然我们已经掌握了从零开始实现矩阵乘法的方法，现在就可以仅使用矩阵乘法来构建神经网络——具体而言，就是实现其前向传播和反向传播过程。

前向传播和反向传播

正如我们在第4章所见，要训练模型，我们需要计算给定损失函数对所有参数的梯度，这被称为反向传播。在正向传播中，我们基于矩阵乘法计算模型对特定输入的输出。在定义首个神经网络时，我们将深入探讨权重初始化的关键问题——这对于确保训练正常启动至关重要。

定义并初始化一层

我们将首先以两层神经网络为例。正如我们所见，单层网络可表示为 $y = x @ w + b$ ，其中 x 是输入， y 是输出， w 是该层的权重（若不进行转置，其大小为输入数量乘以神经元数量）， b 是偏置向量：

```
def lin(x, w, b): return x @ w + b
```

我们可以将第二层叠加在第一层之上，但由于数学上两个线性运算的复合仍是线性运算，这只有在中间加入非线性元素（称为激活函数）时才有意义。正如本章开头所述，在深度学习应用中，最常用的激活函数是ReLU，它返回 x 与 0 中的较大值。

本章不会实际训练模型，因此我们将使用随机张量作为输入和目标。假设输入是200个大小为100的向量，我们将其分组为一个批次；目标则是200个随机浮点数：

```
x = torch.randn(200, 100)
y = torch.randn(200)
```

对于我们的两层模型，需要两个权重矩阵和两个偏置向量。假设隐藏层大小为50，输出层大小为1（在本示例中，每个输入对应一个浮点数输出）。权重初始化为随机值，偏置初始化为零：

```
w1 = torch.randn(100, 50)
b1 = torch.zeros(50)
w2 = torch.randn(50, 1)
b2 = torch.zeros(1)
```

那么我们第一层的结果就是这样：

```
l1 = lin(x, w1, b1)
l1.shape
```

```
torch.Size([200, 50])
```

请注意，该公式适用于我们的输入批次，并返回一个隐藏状态批次：`l1` 是尺寸为 200（我们的批次大小） $\times$ 50（我们的隐藏层尺寸）的矩阵。

然而，我们的模型初始化方式存在问题。要理解这个问题，我们需要查看 `l1` 的均值和标准差（std）：

```
l1.mean(), l1.std()
```

```
(tensor(0.0019), tensor(10.1058))
```

均值接近零，这很正常，因为我们的输入矩阵和权重矩阵的均值都接近零。但标准差——它代表激活值偏离均值的程度——却从1飙升至10。这确实是个大问题，因为这仅仅发生在单层网络中。现代神经网络可能包含数百层，若每层都将激活值的尺度放大10倍，到最后一层时，我们得到的数值将超出计算机的表示能力。

事实上，只要对 x 与 100×100 尺寸的随机矩阵进行50次乘法运算，就会得到这样的结果：

```
x = torch.randn(200, 100)
for i in range(50): x = x @ torch.randn(100,100)
x[0:5,0:5]
```

```
tensor([[nan, nan, nan, nan, nan],
        [nan, nan, nan, nan, nan],
        [nan, nan, nan, nan, nan],
        [nan, nan, nan, nan, nan],
        [nan, nan, nan, nan, nan]])
```

结果到处都是 `nan`。也许我们的矩阵规模过大，需要更小的权重？但若使用过小的权重，又会出现相反的问题——激活值的范围将从1缩小到0.1，经过100层后，最终得到的是满屏的零：

```
x = torch.randn(200, 100)
for i in range(50): x = x @ (torch.randn(100,100) * 0.01)
x[0:5,0:5]
```

```
tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]])
```

因此我们必须精确调整权重矩阵的缩放因子，确保激活值的标准差保持为1。正如Xavier Glorot和Yoshua Bengio在[《理解深度前馈神经网络训练的难度》](#)中所阐述的，可通过数学方法精确计算该值：给定层的正确缩放因子为 $1/\sqrt{n_{in}}$ ，其中 n_{in} 代表输入数量。

在我们的情况下，如果输入有100个，我们应该将权重矩阵缩放为0.1：

```
x = torch.randn(200, 100)
for i in range(50): x = x @ (torch.randn(100,100) * 0.1)
x[0:5,0:5]
```

```
tensor([[ 0.7554, 0.6167, -0.1757, -1.5662, 0.5644],
        [-0.1987, 0.6292, 0.3283, -1.1538, 0.5416],
        [ 0.6106, 0.2556, -0.0618, -0.9463, 0.4445],
        [ 0.4484, 0.7144, 0.1164, -0.8626, 0.4413],
        [ 0.3463, 0.5930, 0.3375, -0.9486, 0.5643]])
```

终于出现了一些既非零也 `nan` 的数值！请注意我们的激活值尺度有多么稳定，即使经过50个虚假层之后依然如此：

```
x.std()
```

```
tensor(0.7042)
```

若稍作调整缩放值，你会发现即使从0.1稍作偏离，也会导致结果呈现极小或极大的数值，因此正确初始化权重至关重要。

让我们回到我们的神经网络。由于我们稍微调整了输入，需要重新定义它们：

```
x = torch.randn(200, 100)
y = torch.randn(200)
```

至于权重，我们将采用正确的权重初始化方法，即Xavier初始化（或称Glorot初始化）：

```
from math import sqrt
w1 = torch.randn(100,50) / sqrt(100)
b1 = torch.zeros(50)
w2 = torch.randn(50,1) / sqrt(50)
b2 = torch.zeros(1)
```

现在如果我们计算第一层的结果，可以验证均值和标准差都在可控范围内：

```
l1 = lin(x, w1, b1)
l1.mean(), l1.std()
```

```
(tensor(-0.0050), tensor(1.0000))
```

很好。现在我们需要经过一个ReLU激活函数，因此来定义一个。ReLU会移除负值并用零替换它们，这意味着它将张量值限制在零点：

```
def relu(x): return x.clamp_min(0.)
```

我们通过此方式传递激活：

```
l2 = relu(l1)
l2.mean(), l2.std()
```

```
(tensor(0.3961), tensor(0.5783))
```

我们又回到了起点：激活值的均值已降至0.4（这很正常，毕竟我们去除了负值），标准差也降至0.58。因此和之前一样，经过几层处理后，最终结果很可能全是零：

```
x = torch.randn(200, 100)
for i in range(50): x = relu(x @ (torch.randn(100,100) *
0.1))
x[0:5,0:5]
```

```
tensor([[0.0000e+00, 1.9689e-08, 4.2820e-08, 0.0000e+00,
0.0000e+00],
[0.0000e+00, 1.6701e-08, 4.3501e-08, 0.0000e+00,
0.0000e+00],
[0.0000e+00, 1.0976e-08, 3.0411e-08, 0.0000e+00,
0.0000e+00],
[0.0000e+00, 1.8457e-08, 4.9469e-08, 0.0000e+00,
0.0000e+00],
[0.0000e+00, 1.9949e-08, 4.1643e-08, 0.0000e+00,
0.0000e+00]])
```

这意味着我们的初始化设置存在问题。原因何在？当Glorot和Bengio撰写论文时，神经网络中最常用的激活函数是双曲正切函数（tanh，即他们采用的函数），而该初始化方案并未考虑ReLU特性。所幸已有研究者完成了相关计算，为我们推导出了正确的缩放因子。在[《深入研究整流器：超越人类水平的性能》](#)（即我们之前提到的引入ResNet的论文）中，Kaiming He等人指出应改用以下缩放因子： $\sqrt{2/n_{in}}$ ，其中 n_{in} 是模型的输入数量。让我们看看这会带来什么效果：

```
x = torch.randn(200, 100)
for i in range(50): x = relu(x @ (torch.randn(100,100) *
sqrt(2/100)))
x[0:5,0:5]
```

```
tensor([[0.2871, 0.0000, 0.0000, 0.0000, 0.0026],
[0.4546, 0.0000, 0.0000, 0.0000, 0.0015],
[0.6178, 0.0000, 0.0000, 0.0180, 0.0079],
[0.3333, 0.0000, 0.0000, 0.0545, 0.0000],
[0.1940, 0.0000, 0.0000, 0.0000, 0.0096]])
```

这次情况好多了：我们的数值这次没有全部归零。那么让我们回到神经网络的定义，并使用这种初始化方法（它被称为Kaiming 初始化或 He 初始化）：

```
x = torch.randn(200, 100)
y = torch.randn(200)

w1 = torch.randn(100,50) * sqrt(2 / 100)
b1 = torch.zeros(50)
w2 = torch.randn(50,1) * sqrt(2 / 50)
b2 = torch.zeros(1)
```

让我们看看经过第一层线性层和ReLU激活后的激活值规模：

```
l1 = lin(x, w1, b1)
l2 = relu(l1)
l2.mean(), l2.std()
```

```
(tensor(0.5661), tensor(0.8339))
```

好多了！现在权重已正确初始化，我们可以定义整个模型：

```
def model(x):
    l1 = lin(x, w1, b1)
    l2 = relu(l1)
    l3 = lin(l2, w2, b2)
    return l3
```

这是前向传播。现在只需将我们的输出与现有标签（本例中为随机数）通过损失函数进行比较。此处我们将采用均方误差（MSE）。（这是个简化问题，而该损失函数最便于后续计算梯度。）

唯一的微妙之处在于我们的输出和目标并非完全同形——经过模型处理后，我们得到的是这样的输出：

```
out = model(x)
out.shape
```

```
torch.Size([200, 1])
```

为消除这个尾随的1维度，我们使用 `squeeze` 函数：

```
def mse(output, targ): return (output.squeeze(-1) -
targ).pow(2).mean()
```

现在我们可以计算损失了：

```
loss = mse(out, y)
```

关于前向传播就说到这里——现在来看看梯度。

梯度与反向传播

我们已经看到，PyTorch通过对 `loss.backward` 的魔法调用计算出我们所需的所有梯度，但让我们深入探究其背后的运作机制。

现在需要计算模型所有权重（即 `w1`、`b1`、`w2` 和 `b2` 中的所有浮点数）对损失函数的梯度。为此需要运用一些数学知识——具体来说是链式法则。这是微积分中指导我们如何计算复合函数导数的规则：

$$(g \circ f)'(x) = g'(f(x))f'(x)$$

JEREMY说

我发现这个符号很难理解，所以更倾向于这样思考：如果 `y = g(u)` 且 `u = f(x)`，那么 `dy/dx = dy/du * du/dx`。这两种符号表示相同的概念，因此选择适合你的即可。

我们的损失函数是由多种功能组成的复合体：均方误差（其本身又是均值与2的幂次的组合）、第二层线性层、ReLU激活函数以及第一层线性层。例如，若要求解损失函数对 `b2` 的梯度，且我们的损失函数定义如下：

```
loss = mse(out,y) = mse(lin(l2, w2, b2), y)
```

链式法则告诉我们，我们有以下公式：

$$\frac{dloss}{db_2} = \frac{dloss}{dout} \times \frac{dout}{db_2} = \frac{d}{dout} mse(out, y) \times \frac{d}{db_2} lin(l_2, w_2, b_2)$$

要计算损失函数对 b_2 的梯度，我们首先需要计算损失函数对输出 `out` 的梯度。若要求解损失函数对 w_2 的梯度，过程同样适用。因此，要获得损失函数对 b_1 或 w_1 的梯度，我们需要先求解损失函数对 l_1 的梯度，而这又需要损失函数对 l_2 的梯度，进而需要损失函数对输出 `out` 的梯度。

因此，要计算更新所需的所有梯度，我们需要从模型的输出开始，逐层向后推导——这就是该步骤被称为反向传播的原因。我们可通过让每个实现的函数（`relu`、`mse`、`lin`）提供其反向传播步骤来自动化该过程：即如何从损失函数对输出的梯度推导出损失函数对输入的梯度。

在此我们将这些梯度填充到每个张量的属性中，有点类似于PyTorch对 `.grad` 的处理方式。

首先是损失函数对模型输出（即损失函数输入）的梯度。我们先撤销 `mse` 中的 `squeeze` 操作，然后使用求导公式得到 x^2 的导数： $2x$ 。均值的导数即为 $1/n$ ，其中 n 是输入元素的数量：

```
def mse_grad(inp, targ):
    # grad of loss with respect to output of previous layer
    inp.g = 2. * (inp.squeeze() - targ).unsqueeze(-1) /
    inp.shape[0]
```

对于ReLU层和线性层的梯度，我们使用损失函数对输出（即 `out.g`）的梯度，并应用链式法则计算损失函数对输入（即 `inp.g`）的梯度。链式法则表明：`inp.g = relu'(inp) * out.g`。`relu` 函数的导数取值为0（当输入为负值时）或1（当输入为正值时），因此可得：

```
def relu_grad(inp, out):
    # grad of relu with respect to input activations
    inp.g = (inp>0).float() * out.g
```

该方案用于计算损失函数对线性层中输入、权重和偏置的梯度，其原理如下：

```
def lin_grad(inp, out, w, b):
    # grad of matmul with respect to input
    inp.g = out.g @ w.t()
    w.g = inp.t() @ out.g
    b.g = out.g.sum(0)
```

我们不会过多探讨定义这些公式的数学表达式，因为它们对我们的目的而言并不重要。不过，如果你对这个主题感兴趣，不妨看看可汗学院精彩的微积分课程。

SymPy

SymPy 是一个符号计算库，在处理微积分时极为实用。根据 [文档](#) 所述：

符号计算处理的是数学对象的符号化计算。这意味着数学对象被精确表示而非近似表示，且含有未求值变量的数学表达式保持符号形式。

要进行符号计算，我们先定义一个符号，然后进行计算，如下所示：

```
from sympy import symbols, diff
sx, sy = symbols('sx sy')
diff(sx**2, sx)
```

```
2*sx
```

这里，SymPy已经为我们求出了 `x**2` 的导数！它能处理复杂的复合表达式求导，简化并因式分解方程，而且它的功能远不止于此。如今人们几乎没有必要手动进行微积分计算——计算梯度时PyTorch代劳，展示方程时SymPy代劳！

定义完这些函数后，我们便可利用它们编写反向传播。由于每个梯度值会自动填入对应张量中，我们无需将 `_grad` 函数的结果存储在任何地方——只需按正向传播的逆序执行这些函数，确保每个函数中都存在 `out.g`：

```
def forward_and_backward(inp, targ):
    # forward pass:
    l1 = inp @ w1 + b1
    l2 = relu(l1)
    out = l2 @ w2 + b2
    # we don't actually need the loss in backward!
    loss = mse(out, targ)
    # backward pass:
    mse_grad(out, targ)
    lin_grad(l2, out, w2, b2)
    relu_grad(l1, l2)
    lin_grad(inp, l1, w1, b1)
```

现在我们可以访问模型参数的梯度，分别存储在 `w1.g`、`b1.g`、`w2.g` 和 `b2.g` 中。我们已成功定义模型——接下来让我们将其改造成更符合 PyTorch 模块的形态。

重构模型

我们使用的三个函数关联着两个操作：正向传播和反向传播。无需分别编写这两个过程，我们可以创建一个类将它们封装起来。该类还能存储反向传播所需的输入和输出参数。这样，我们只需调用 `backward` 函数即可：

```
class Relu():
    def __call__(self, inp):
        self.inp = inp
        self.out = inp.clamp_min(0.)
        return self.out

    def backward(self): self.inp.g = (self.inp>0).float() * self.out.g
```

`__call__` 是 Python 中的一个魔术名称，它将使我们的类可调用。当我们输入 `y = Relu() (x)` 时，这段代码就会被执行。我们也可以为线性层和均方误差损失函数实现类似功能：

```
class Lin():
    def __init__(self, w, b): self.w, self.b = w, b
```



```

def __call__(self, inp):
    self.inp = inp
    self.out = inp@self.w + self.b
    return self.out

def backward(self):
    self.inp.g = self.out.g @ self.w.t()
    self.w.g = self.inp.t() @ self.out.g
    self.b.g = self.out.g.sum(0)

class Mse():
    def __call__(self, inp, targ):
        self.inp = inp
        self.targ = targ
        self.out = (inp.squeeze() - targ).pow(2).mean()
        return self.out

    def backward(self):
        x = (self.inp.squeeze()-self.targ).unsqueeze(-1)
        self.inp.g = 2.*x/self.targ.shape[0]

```

然后我们可以将所有内容放入一个模型中，该模型由我们的张量 `w1`、`b1`、`w2` 和 `b2` 初始化：

```

class Model():
    def __init__(self, w1, b1, w2, b2):
        self.layers = [Lin(w1,b1), Relu(), Lin(w2,b2)]
        self.loss = Mse()

    def __call__(self, x, targ):
        for l in self.layers: x = l(x)
        return self.loss(x, targ)

    def backward(self):
        self.loss.backward()
        for l in reversed(self.layers): l.backward()

```

这种重构方式的优点在于，将事物注册为模型层后，正向和反向传播的实现变得非常简单。若需实例化模型，只需编写如下代码：

```
model = Model(w1, b1, w2, b2)
```

前向传播可按以下方式执行：

```
loss = model(x, y)
```

以及使用此方法的反向传递：

```
model.backward()
```

转向PyTorch

我们编写的 `Lin`、`Mse` 和 `Relu` 类具有许多共同点，因此我们可以让它们都继承自同一个基类：

```
class LayerFunction():
    def __call__(self, *args):
        self.args = args
        self.out = self.forward(*args)
        return self.out

    def forward(self): raise Exception('not implemented')
    def bwd(self): raise Exception('not implemented')
    def backward(self): self.bwd(self.out, *self.args)
```

然后我们只需在每个子类中实现 `forward` 和 `bwd` 方法：

```
class Relu(LayerFunction):
    def forward(self, inp): return inp.clamp_min(0.)
    def bwd(self, out, inp): inp.g = (inp>0).float() * out.g

class Lin(LayerFunction):
    def __init__(self, w, b): self.w,self.b = w,b

    def forward(self, inp): return inp@self.w + self.b

    def bwd(self, out, inp):
        inp.g = out.g @ self.w.t()
        self.w.g = self.inp.t() @ self.out.g
        self.b.g = out.g.sum(0)

class Mse(LayerFunction):
    def forward (self, inp, targ): return (inp.squeeze() - targ).pow(2).mean()

    def bwd(self, out, inp, targ):
        inp.g = 2*(inp.squeeze()-targ).unsqueeze(-1) / targ.shape[0]
```

模型其余部分可与之前保持一致。这正越来越接近 PyTorch 的实现方式。每个需要求导的基本函数都以 `torch.autograd.Function` 对象的形式编写，该对象包含 `forward` 和 `backward` 方法。PyTorch 会追踪所有计算操作，以便正确执行反向传播——除非我们将张量的 `requires_grad` 属性设为 `False`。

编写这类函数（几乎）和编写原始类一样简单。区别在于我们需要选择保存的内容以及放入上下文变量的内容（以确保不保存任何不需要的内容），并在 `backward` 中返回梯度。虽然很少需要编写自定义函数，但若遇到特殊需求或想调整常规函数的梯度，可按以下方式实现：

```

from torch.autograd import Function
class MyRelu(Function):
    @staticmethod
    def forward(ctx, i):
        result = i.clamp_min(0.)
        ctx.save_for_backward(i)
        return result

    @staticmethod
    def backward(ctx, grad_output):
        i, = ctx.saved_tensors
        return grad_output * (i>0).float()

```

用于构建更复杂模型以利用这些函数的结构是 `torch.nn.Module`。这是所有模型的基础结构，迄今为止你所见到的所有神经网络都属于该类。它主要用于注册所有可训练参数，正如我们所见，这些参数可在训练循环中使用。

要实现 `nn.Module`，只需执行以下步骤：

1. 确保在初始化时首先调用父类的 `__init__` 方法。
2. 使用 `nn.Parameter` 定义模型的所有参数属性。
3. 定义一个 `forward` 函数，该函数返回模型的输出结果。

例如，以下是从零开始构建的线性层：

```

import torch.nn as nn

class LinearLayer(nn.Module):
    def __init__(self, n_in, n_out):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(n_out, n_in)
                                    * sqrt(2/n_in))
        self.bias = nn.Parameter(torch.zeros(n_out))

    def forward(self, x): return x @ self.weight.t() + self.bias

```

如你所见，本类会自动记录哪些参数已被定义：

```

lin = LinearLayer(10,2)
p1,p2 = lin.parameters()
p1.shape,p2.shape

```

```

(torch.Size([2, 10]), torch.Size([2]))

```

正是由于 `nn.Module` 的这一特性，我们只需声明 `opt.step`，优化器便会自动遍历所有参数并逐个更新。

请注意，在 PyTorch 中，权重以 `n_out x n_in` 的矩阵形式存储，因此我们在正向传播中需要进行转置操作。

通过使用 PyTorch 的线性层（该层同样采用 Kaiming 初始化），本章构建的模型可表示为：

```
class Model(nn.Module):
    def __init__(self, n_in, nh, n_out):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(n_in,nh), nn.ReLU(),
            nn.Linear(nh,n_out))
        self.loss = mse
    def forward(self, x, targ): return self.loss(self.layers(x).squeeze(), targ)
```

fastai 提供了自己的 `Module` 变体，它与 `nn.Module` 完全相同，但无需你调用 `super().__init__()`（它会自动为你完成）：

```
class Model(Module):
    def __init__(self, n_in, nh, n_out):
        self.layers = nn.Sequential(
            nn.Linear(n_in,nh), nn.ReLU(),
            nn.Linear(nh,n_out))
        self.loss = mse
    def forward(self, x, targ): return self.loss(self.layers(x).squeeze(), targ)
```

在第19章中，我们将从这样的模型出发，学习如何从零构建训练循环，并将其重构为我们在前几章中使用形式。

总结

在本章中，我们探索了深度学习的基础，从矩阵乘法开始，逐步实现神经网络的前向传播和反向传播。随后我们重构了代码，展示了PyTorch底层的工作原理。

以下是几点需要记住的事项：

- 神经网络本质上是由矩阵乘法组成的集合，其间穿插着非线性操作。
- Python运行较慢，因此要编写高效代码，我们必须进行向量化处理，并利用元素级运算和广播等技术。
- 若两个张量的末端开始向后递归的维度匹配（即维度相同或其中一个为1），则它们可进行广播。为使张量具备广播性，可能需要通过 `unsqueeze` 或 `None` 索引添加1维度。
- 正确初始化神经网络对启动训练至关重要。当使用ReLU非线性函数时，应采用Kaiming初始化。
- 反向传播是链式法则的多次应用，从模型输出逐层逆向计算梯度。
- 当继承 `nn.Module` 类（若未使用 fastai 的 `Module`）时，需在 `__init__` 方法中调用父类的 `__init__` 方法，并定义接收输入并返回预期结果的 `forward` 函数。

问卷调查

1. 编写实现单个神经元的Python代码。
2. 编写实现ReLU函数的Python代码。
3. 编写基于矩阵乘法的全连接层Python代码。
4. 编写纯Python实现的全连接层代码（即使用列表推导式和Python内置功能）。
5. 什么是层的“隐藏层大小”？
6. PyTorch 中 `t` 方法的作用是什么？
7. 为什么纯 Python 实现的矩阵乘法效率极低？

8. 在 `matmul` 中, 为何 `ac==br` ?
9. 在 Jupyter Notebook 中如何测量单个单元格的执行时间?
10. 什么是元素级运算?
11. 编写 PyTorch 代码, 测试 `a` 的每个元素是否大于 `b` 的对应元素。
12. 什么是阶数为 0 的张量? 如何将其转换为普通 Python 数据类型?
13. 此代码返回什么结果? 原因是什么?

```
tensor([1,2]) + tensor([1])
```

14. 此表达式返回什么结果? 原因?

```
tensor([1,2]) + tensor([1,2,3])
```

15. 元素级运算如何帮助我们加速矩阵乘法?
16. 广播规则是什么?
17. `expand_as` 是什么? 举例说明如何使用它来匹配广播结果。
18. `unsqueeze` 如何帮助解决特定广播问题?
19. 如何通过索引实现与 `unsqueeze` 相同的效果?
20. 如何展示张量实际使用的内存内容?
21. 当向 3×3 矩阵添加 3 维向量时, 向量元素会被添加到矩阵的每行还是每列? 向量元素是与矩阵的每行还是每列相加? (请务必在笔记本中运行代码验证答案)
22. 广播和 `expand_as` 是否会增加内存使用? 为什么?
23. 使用爱因斯坦求和实现矩阵乘法。
24. `einsum` 左侧的重复索引字母代表什么含义?
25. 爱因斯坦求和符号的三大规则是什么? 为什么?
26. 神经网络的前向传播与反向传播分别指什么?
27. 为何需要存储前向传播中计算的中间层激活值?
28. 激活值标准差偏离 1 过远会产生什么弊端?
29. 权重初始化如何帮助避免此问题?
30. 对于纯线性层及线性层后接 ReLU 层, 分别如何初始化权重以获得标准差为 1 的结果?
31. 为何损失函数中需采用 `squeeze` 方法?
32. `squeeze` 方法的参数作用是什么? 尽管 PyTorch 不强制要求, 为何添加该参数可能至关重要?
33. 什么是链式法则? 用本章给出的两种形式之一展示其公式。
34. 展示如何运用链式法则计算 `mse(lin(l2, w2, b2), y)` 的梯度。
35. ReLU 函数的梯度是什么? 用数学表达式或代码展示。(无需死记硬背——尝试根据函数形状推导)
36. 反向传播中需按什么顺序调用 `*_grad` 函数? 原因?
37. `__call__` 是什么?
38. 实现 `torch.autograd.Function` 时必须实现哪些方法?
39. 从零编写 `nn.Linear` 并测试其功能。
40. `nn.Module` 与 fastai 的 `Module` 有何区别?

进一步研究

1. 将ReLU实现为 `torch.autograd.Function`，并用其训练模型。
2. 若具备数学背景，请用数学符号推导线性层的梯度。将其映射到本章的实现中。
3. 学习PyTorch中的 `unfold` 方法，结合矩阵乘法实现自定义的二维卷积函数，并训练使用该函数的卷积神经网络。
4. 使用NumPy替代PyTorch实现本章所有内容。